

Nourally. A complete hands-on review.

What works. What breaks.
How to make the app clearer, faster,
and more polished.

23 prioritized findings	4 viewport widths
6 primary destinations	Live USDA + barcode lookups

September 14, 2026

Includes reproduced bugs, screen-by-screen recommendations, a connected-food data model, an ordered implementation backlog, test coverage, and visual evidence. No app features changed during this audit.

Contents

01 / Executive verdict	3
02 / Method, environment, and evidence quality	4
03 / What already works and should be preserved	5
04 / Priority map: fix these first	6
05 / Food identity and dietary trust	7
06 / Grocery generation, budget, and purchase integrity	8
07 / Accounts and saved-data reliability	9
08 / Food portions, logs, and barcode consistency	10
09 / Schedule timing and next-action correctness	11
10 / Reminders and preparation lists	12
11 / Food workspace: the largest usability problem	13
12 / Search quality and database integration	14
13 / Pantry and meal suggestions: make the connection practical	15
14 / Schedule editing and calendar usability	16
15 / Today: make the first screen actionable	17
16 / A sleeker visual system	18
17 / Navigation, responsive behavior, and keyboard access	19
18 / Weekly, History, Profile, and small consistency issues	20
19 / The connected food record: implementation design	21
20 / The desired end-to-end food workflow	22
21 / Backend, catalog, and resilience work	23
22 / What to add, what to change, and what to defer	24
23 / Implementation sequence and acceptance gates	25
24 / Functional test matrix: completed paths	26
25 / Functional test matrix: food, reliability, and limits	27
26 / Regression suite to build from this audit	28
27 / Evidence index and source pointers	29
28 / Standards, open questions, and final recommendation	30
Visual A / Today at an intermediate width	31
Visual B / Mobile Food: the selected task is buried	32
Visual C / Mobile calendar and weekly summary	33
Visual D / Mobile Today and Profile	34

01 / Executive verdict

September 14, 2026 | Functional testing, product workflow, visual design, and implementation recommendations

Nourally has a useful core: turn a school-and-sport schedule into a practical food plan. I used the running application, entered a synthetic athlete's schedule, searched real food databases, stocked a pantry, generated groceries, moved items through the cart, logged food, changed dietary settings, and inspected the results at multiple screen sizes. This is a hands-on audit of the current local build, not a review of a concept document.

The strongest completed connection is **USDA search** → **grocery item** → **purchase** → **pantry**. A purchased USDA banana correctly increased the existing inventory quantity instead of creating a second identical USDA record. Basic schedule editing, recurring activities, manual logging, hydration, and daily rollover also worked in the tested paths.

The main weakness is that the interface suggests more integration and reliability than the underlying rules currently deliver. Pantry matching can confuse different ingredients. Grocery generation can erase hand-added items and exceed a tiny budget. Food portions are inconsistent across entry methods. Account creation does not isolate data. These problems matter more than adding more food choices or decorative elements.

My recommendation is to spend the next development cycle on three outcomes:

1. **Trust the data:** safe inventory identity, reliable saving, truthful portions and prices, and explicit local-profile behavior.
2. **Shorten the daily workflow:** selected Food content immediately visible, quick contextual food entry, and useful schedule-to-food transitions.
3. **Make the interface feel like a daily app:** smaller repeated headings, consistent navigation, denser lists, stronger text legibility, and fewer nested surfaces.

The current palette and name are worth keeping. A sleek redesign does not require photographs, a new brand, more tabs, or a dashboard full of metrics. It requires a clearer hierarchy and less work between an intention and its completion.

Scope boundary: No application features were changed during this audit. Test data was created in an isolated browser session, not in the user's existing in-app browser. This report contains app work only: no publishing, interview, outreach, or screenshot-sending assignments.

02 / Method, environment, and evidence quality

The audit ran against `http://127.0.0.1:5173/` in the local project. The server was initially unavailable; I started the Vite development server and used a separate Chromium session named `nourally-audit`. The production build also completed successfully.

Screen sizes tested: 390 x 844, 768 x 1000, 1024 x 1000, and 1440 x 1000 CSS pixels. These are browser viewport checks, not physical-device certification. All six primary destinations were visited at each width. Food's five subsections were exercised during the functional walkthrough.

Test fixture: Audit Athlete; a weekday school schedule from 8:00 AM to 3:00 PM; lunch 11:30 AM to noon; snack windows; a 7:00 PM away practice with 45 minutes of travel; an early recurring away game; pantry items; purchases; three food logs; and hydration entries. Tests also used isolated copies of that fixture for account changes, malformed storage, and controlled-clock scenarios.

Evidence is classified throughout the report:

- **Observed:** reproduced by interacting with the running browser and reading its rendered state.
- **Controlled:** reproduced in the browser with a synthetic clock, permission response, media stream, network response, or saved-data fixture. The production UI and application logic still ran.
- **Source:** established by reading the current implementation; not a claim of a separately executed end-to-end path.
- **Proposal:** a suggested change, not an existing feature or measured user outcome.

Real USDA searches and a real Open Food Facts barcode lookup succeeded. The 429 response, zero-calorie barcode product, no-result response, camera-denial response, and notification permission responses were deliberately simulated. Do not interpret those fixtures as live provider failures.

Not verified: physical phone-camera autofocus and barcode recognition, real operating-system notification delivery, closed-app reminders, Safari/Firefox behavior, installed-PWA behavior, production hosting, real authentication infrastructure, multi-device sync, a security penetration test, or expert validation of nutrition advice. No payment or retailer purchase was made.

The evidence folder contains screenshots, browser snapshots, and small reproducible audit scripts. A successful build is useful but does not replace functional tests. The repository currently has no dedicated test, lint, or typecheck script.

03 / What already works and should be preserved

There is a substantial functioning foundation here. The next iteration should not remove useful features merely to make the page shorter.

Schedule entry is more complete than a basic calendar. School dates, weekdays, school hours, lunch, snack windows, food access, commute, activity type, intensity, location, travel duration, weekly repetition, and individual skipped occurrences are represented. School cancellation and restoration worked. A one-time practice could be edited. A travel workout could be created and deleted. A weekly series could be edited, skipped on one date, and deleted.

Several sensible input checks already exist. End-before-start activity times are rejected. Lunch must fit inside the school day. Snack windows outside school are rejected. A recurring event needs at least one weekday. A profile cannot be saved after all food-access sources are deselected. Preserve these checks while consolidating form validation.

Food entry has real provider connections. Generic banana searches returned Foundation foods. Branded Cheerios searches returned branded records. All-food peanut-butter search also returned survey food. The app is not just searching its small grocery catalog. The barcode fallback returned Nutella from Open Food Facts and correctly calculated 81 kcal for a 15 g entry from a 539 kcal-per-100 g result.

Local planning interactions work. Pantry quantity adjustment, case-insensitive manual pantry merging, custom groceries, list-to-cart, cart-to-list, marking purchased, purchase history, and adding missing meal ingredients all worked in basic cases. Repeating the same missing-ingredient action did not duplicate the already queued Pretzels item. Buying Pretzels changed Banana + pretzels from 1/2 to 2/2 ingredients available.

Daily records and visual identity have good foundations. Manual food add/edit/delete worked. Hydration increments and the explicitly labeled 8 oz subtraction worked. Reload preserved the ordinary single-tab fixture. A controlled midnight rollover reset today's water and moved the prior day's three food entries into History. The dark teal, lime action accent, and restrained editorial headline make Nourally recognizable.

Keep the performance-positive wording and the distinction between a record of check-ins and a prescribed intake target. Improve precision and convenience around those foundations instead of expanding into a calorie-goal product.

04 / Priority map: fix these first

Severity here reflects product impact, not a formal security rating. **P1** means high-priority correctness, data integrity, or access-boundary work. **P2** means significant workflow, clarity, or resilience work. **P3** means polish. There is no claim that every P1 issue is remotely exploitable.

ID	Priority	Finding	Evidence
B01	P1	Peanut butter satisfies sunflower-butter availability	Controlled
B02	P1	Generating groceries removes custom list items	Observed
B03	P1	A new local account inherits existing food and activity data	Observed, isolated context
B04	P1	Two tabs overwrite one another's saved updates	Controlled two-tab test
B05	P1	Malformed saved JSON causes a blank application	Controlled
B06	P1	Zero/tiny budgets still generate unaffordable lists	Observed
B07	P1	Unknown USDA prices count as zero in the budget	Observed
B08	P1	Portions and manual overrides lose nutritional meaning	Observed + source
B09	P2	Food subtab content is buried below search results	Observed
B10	P2	Current lunch window is skipped after its start	Controlled clock
B11	P2	Upcoming activity suppresses just-finished recovery context	Controlled clock
B12	P2	Reminder title reports configured lead, not time remaining	Controlled notification
B13	P2	No matching meal options leads to an unexplained empty panel	Controlled
B14	P2	USDA and catalog identities create duplicate grocery recommendations	Observed
B15	P2	Search errors can leave stale actionable results	Observed + controlled
B16	P2	Generic search only includes Foundation; results stop at 18	Observed + source
B17	P2	Skipped recurring sports days cannot be restored in the UI	Observed + source
B18	P2	Future grocery-trip dates save and appear as Today	Observed
B19	P2	Zero-calorie barcode entries are blocked	Controlled
B20	P2	Barcode maximum portion is not enforced by its Add action	Observed control state + source
B21	P2	Tiny targets, weak labels, and incomplete selected-state semantics	Measured + source
B22	P2	Private USDA key would be exposed in the browser	Source + official docs
B23	P3	Missing favicon produces a 404	Observed console

Fix B01-B08 before widening the catalog or adding more automation. Fix B09 early in the same cycle: it makes the already working functionality much easier to use. The later chapters describe scope and acceptance criteria so these can become development tasks rather than vague complaints.

05 / Food identity and dietary trust

B01: Different ingredients are treated as equivalent

Reproduction: In an isolated fixture, set a nut-free preference and stock Peanut butter, Whole-grain bread, and Bananas. Open Food > Meals before a practice and expand more options. The Sunflower-butter banana sandwich reports **3/3 at home**, even though no sunflower-seed butter was added.

Cause: `pantryHasIngredient` in `src/main.jsx:745` considers a match when any long-enough keyword overlaps. Both ingredient names contain “butter.” An ingredient availability check is therefore behaving like a broad text search.

Impact: The app confidently reports that a required ingredient is present when a different food is present. A disclaimer does not fix incorrect identity. The audit did not test consumption or make a clinical outcome claim; the reproduced defect is the false availability assertion itself.

Fix: Separate discovery from identity. Search may be fuzzy. Pantry satisfaction should require an approved ingredient relationship: an exact canonical ingredient, a deliberately defined alias, or a user-approved substitute. “Peanut butter” must never silently stand in for “sunflower-seed butter.” Unknown matches should be unresolved, not optimistically accepted.

Acceptance: Tests for peanut versus sunflower butter, dairy versus soy milk, raw rice versus rice cakes, beans versus unrelated foods containing “bean,” and dietary-incompatible substitutes. Show the actual pantry item behind each “Have it” indicator. Let the user correct a proposed match.

B14: The reverse problem creates unnecessary groceries

A USDA ripe-banana item was already in the pantry and cart, but Generate grocery list still proposed the catalog item Bananas. The generator checks exact names and catalog IDs, while meal availability uses loose keywords. These two different definitions of “same food” explain why one part of the app can overmatch and another can undermatch.

Create one ingredient-resolution service used by pantry, shopping, and meals. Preserve specific product identity separately: a branded cereal and a generic cereal category are related, but are not the same product. Record the relationship rather than changing the source record's name.

Preserve dietary choices without overstating them

Vegan, gluten-free, and nut-free toggles changed the displayed idea set in the walkthrough. That is useful, but the library uses broad hard-coded flags. Mixed alternatives such as “Chicken or tofu rice bowl” should become explicit variants, each with its own ingredients and constraints. Do not infer a specific packaged product's suitability from a generic template's flag.

06 / Grocery generation, budget, and purchase integrity

B02: Regeneration destroys user-authored work

Reproduction: Add a custom grocery called Keep my special granola with a \$7.50 estimate. Generate a list. The custom item disappears; existing cart items are preserved. This happened without a replacement warning or undo.

Cause: generateList, src/main.jsx:858, replaces items with the current cart plus newly chosen catalog candidates. Existing items with list status are discarded regardless of who added them.

Fix: Track item origin: manual, search, meal requirement, or generated. Default generation to a preview that merges new recommendations and keeps manual/search items. Offer a separate, explicit replacement action if needed. Preserve quantities, checked state, prices, and notes for retained items.

Acceptance: A custom item, a USDA item, a meal requirement, and an existing cart item all survive refresh. Removed recommendations do not reappear without explanation. A user can undo the generated change.

B06: Budget does not behave like a budget

A \$5 budget produced an \$8 list. A \$0 budget produced a \$49.25 list. The loop always allows at least three recommended items before applying its overspend rule; zero also disables the limit. The resulting total is labeled over-budget, but the generator still presents the list as its result.

Decide what zero means. If it means no spending available, recommend using current food and show an empty affordable list. If the user wants no limit, provide an explicit No budget set option. For insufficient budgets, show the most useful affordable subset and explain any unresolved needs. Never disguise a minimum basket rule as a budget-constrained recommendation.

B07: Unknown prices are not free

USDA foods enter groceries with price 0. Their rows say Store price not set, but summary totals treat them as free and claim the full remaining balance. There is no row-level price editor. Store price: null, calculate a known subtotal, and show “3 items need prices” beside it. Let users enter a package price and package size before relying on the budget.

Clarify the accounting period

The summary says Weekly budget while the input says Amount for this shop. Purchased items leave the planned total, so the displayed remaining amount is not weekly spending remaining. Choose one model and label it consistently. A proper weekly model needs dated purchase transactions; a per-shop model should reset intentionally and should not imply a weekly ledger.

07 / Accounts and saved-data reliability

B03: Account creation does not create a separate data space

Reproduction: With the audit's food and schedule data present, open `?signedOut=1`, create Second Athlete with an example email, and finish setup. The new name appears, but the new profile still has **2 pantry items, 4 grocery items, 3 food logs, and 52 oz water** from the earlier athlete.

The account panel does disclose Prototype account and says no password is collected. That honesty should be preserved. Nevertheless, “private profile” and “Create account” imply a boundary that is not implemented. Root navigation also bypasses the signed-out screen because the query parameter is the only gate. Signing in checks a locally saved email, not identity ownership.

For a local-only version, rename this to a device profile and either support a real profile switcher with separate namespaces or make clear there is exactly one shared device profile. For a real account version, introduce authentication and ownership checks before calling the data private. Do not solve this by clearing existing data when another person presses Create account.

B04: Two tabs lose updates

Start both tabs with 52 oz. Add 8 oz in tab A: it displays 60. Tab B stays at 52. Add 12 in tab B and reload A: the result is **64**, not the expected **72**. The later write overwrites the earlier change because each tab saves its own stale copy of the full record.

Use revision-aware persistence, cross-tab change notifications, and mergeable operations. Hydration should preferably be a sequence of entries rather than one mutable total. Test simultaneous edits to pantry and groceries too; they use similar whole-object persistence, although those particular races were not separately reproduced.

B05: A damaged saved value blanks the app

Setting only the isolated fixture's grocery value to malformed JSON and reloading produced an empty body with a JSON parse exception. Storage parsing has no recovery boundary. Add schema versions, safe parsing, validation, and a visible recovery screen. Quarantine the damaged value and allow backup/export before reset. Do not automatically erase all user data.

Normal single-tab persistence and daily rollover passed. The problem is resilience around that happy path, not a total absence of persistence.

08 / Food portions, logs, and barcode consistency

B08: Portion meaning is inconsistent

USDA result nutrient previews show calories, carbohydrate, and protein without clearly saying that the figures are per 100 g. The neighboring Log button may log a label serving, such as 30 g, or default to 100 g. A user can reasonably mistake the preview for the amount about to be logged.

There is no shared portion-confirmation step. Generic bananas logged as 100 g. The selected Cheerios record also logged as 100 g. Barcode entry has a grams field, while manual entry has a free calorie field. Editing a USDA log changes name/calories but not grams. I changed the Cheerios entry from 359 to 100 kcal; it still displayed USDA and 100 g, although the source snapshot remained unchanged.

Fix: Use one entry editor with amount, unit, optional household measure, calculated nutrients, and provenance. Label estimates explicitly. If a user overrides a value, show User adjusted and preserve the original source value separately. Prefer a practical measure when supported, but never invent a cup-to-gram conversion or treat milliliters as grams without a justified conversion.

Acceptance: Identical food and quantity produce consistent results from search, pantry, barcode, and log editing. Missing nutrients remain unknown rather than zero. Users can record food without calories. A later database update does not retroactively change historical meals.

B19 and B20: Barcode validation is uneven

A controlled Open Food Facts result with a legitimate zero energy value rendered correctly as 0 kcal but disabled Add to today. A food check-in should not require positive calories. Separately, entering 2500 g exceeded the field's declared maximum of 2000, but the Add control remained enabled. The action is outside the form constraint path; enforce the same numeric bounds in the handler.

The real 15 g Nutella calculation passed. Invalid short barcode input showed a clear message. Camera-denial and no-readable-barcode capture paths displayed errors, and the synthetic media stream stopped after capture. Physical camera success remains unverified.

Barcode products currently become a separate log shape rather than the same shared food record used by USDA pantry and grocery actions. Connect provider adapters before adding more scanning features.

09 / Schedule timing and next-action correctness

B10: The current lunch window disappears

Controlled time: 11:45 AM during configured lunch from 11:30 AM to noon, with no sport due. The next-action timing said **Afternoon snack window at 2:30 PM**. It skipped the lunch period still in progress because it checks only whether a window's start time is in the future.

Store and evaluate intervals, not just start timestamps. First check whether a food window is active; otherwise find the next one. Display "Lunch now, until noon." Add tests at one minute before, exactly at start, mid-window, exactly at end, and one minute after. The meal suggestions should also respect whether the current window is actually available.

B11: A later event suppresses recent recovery context

At 3:10 PM, with a session ending at 3:00, the app correctly showed recovery. Add another practice at 6:00 and it instead showed Use this meal window before the rush, focused entirely on the later event. The priority chain checks any upcoming activity before checking recent activity.

This is an information-priority defect, not a claim that the suggested meal itself is necessarily unsuitable. Show both contexts: just finished and next session. A combined recovery-and-next-session plan can be useful, but the completed session should not disappear from reasoning. Build recommendations from a time-ordered day model rather than one mutually exclusive event branch.

Travel exists as metadata, not a departure schedule

Away location and longer travel correctly push toward portable foods and packing tasks. But a practice starting in 19 minutes with 45 minutes of travel still displays a pre-activity food action without a departure warning. The implementation uses travel primarily to favor portability, not to calculate when the athlete must leave.

Add derived departure and preparation times. Use the activity's actual venue/departure context when known, and label assumptions when it is not. Do not claim the athlete is home merely because a home food source is enabled.

Rest-day behavior needs cleaner prioritization

The no-schedule page still leads with a large food recommendation and a PRE-PRACTICE callout. If there is no schedule yet, make Add school or Add activity the primary setup step. If the schedule intentionally contains a rest day, use neutral language rather than presenting missing setup and rest as the same state.

10 / Reminders and preparation lists

B12: Notification timing text is incorrect

With a 60-minute lead selected and practice about 28 minutes away, the synthetic notification payload said **Soccer practice edited in about 60 minutes**. The code uses the configured lead in the title, even when reminders are enabled late in that window.

Compute the title from the actual interval, or use an absolute start time. Consider “Practice at 7:00 PM” more robust than a rounded countdown if delivery may be delayed. A lead preference determines eligibility, not the displayed remaining time. Test at 61, 60, 30, 1, and 0 minutes, after rescheduling, and after enabling reminders partway through the window.

Working behavior: Lead settings changed, evening preparation could be toggled, granted permission enabled reminders, denied permission displayed a clear message, and reminders could be turned off. The implementation also stores deduplication keys.

Important limit: The page itself says reminders work while Nourally is open. That is accurate. There is no verified closed-app delivery path. Do not advertise background reminders until the scheduling/delivery architecture exists and passes actual-device tests. These browser tests verified UI behavior and payloads, not operating-system notifications.

Make preparation tasks feel connected to real meals

Plan Banana + pretzels created a food task and a bag-placement task. Away travel also created a water task. Completion, removal, and percentage updates worked. Tomorrow's early-game list generated food, gear, and travel preparation, and individual tasks could be completed or removed.

The plan currently stores loosely related tasks, not a meal plan with servings, a time, and an event relationship. Picking an alternate adds more preparation tasks rather than replacing the old meal. Changing the profile can leave now-incompatible earlier tasks visible without review. After removing one tomorrow task, Build tomorrow's list stays hidden while any tasks remain.

Add a meal-plan object with event, intended time, selected recipe/template, portions, and linked prep tasks. Offer Replace meal, Rebuild missing prep, and Undo. Keep shared gear tasks shared: replacing a snack should not duplicate a water bottle or remove another meal's essential task.

11 / Food workspace: the largest usability problem

B09: Switching a subtab does not bring its content into view

Food places a large USDA search panel above every subsection. Search results remain mounted as the user switches between At home, Groceries, Meals, and Food log. On desktop, the result area has its own scroll region. On mobile, that maximum height is removed and all 18 results enter the document flow.

In the measured 390-pixel-wide branded-search state, the Food log began roughly **4,380 pixels down the page**. Tapping Food log changes the selected tab, but the user still has to move past pages of unrelated search results. This can feel as if the tab did not work.

The screenshot evidence is particularly clear: the top of the mobile Food page contains a large title, primary navigation, nested subnavigation, and search controls, but no useful content from the chosen Food task. The issue is information order, not simply missing whitespace.

Recommended interaction

Use a compact contextual Add food control inside the selected subsection. It opens a search sheet or dedicated search view with a destination already selected: At home, Grocery list, or Food log. After choosing a result, show the appropriate quantity/portion editor and return to the originating subsection.

If shared search stays permanently visible, keep it to one compact row and collapse results when the destination changes. Preserve the query for convenience, but do not preserve its visual obstruction. Move focus to the new subsection heading and expose a selected state programmatically.

Simplify the Food overview

The overview currently has counts and another explanatory sequence titled ONE CONNECTED RECORD. That is developer/product messaging, not a daily user task. Replace it with a compact useful summary: food running low, next planned meal, grocery items remaining, and the most recent log. Each summary should lead to the corresponding action.

The user previously asked to remove numbered workflow lists and card framing behind the tab switcher. The current Food overview and subtab container reintroduce similar visual patterns. Do not carry those patterns into the redesign. Preserve the functionality, remove the repeated explanation and nested framing.

12 / Search quality and database integration

B15: Search errors leave old results actionable

After a successful 18-result branded search, submitting a one-letter query showed the validation error but retained all 18 old results. A controlled 429 response did the same. The query and visible results can disagree without an explicit “previous results” label.

Associate each result set with its successful query and filter. Clear it on invalid replacement searches, or label it unambiguously as previous results. Disable actions on a stale set while a new search is being resolved if the presentation implies the new query is active. Preserve manual entry when providers fail. Add Retry and a useful no-results action.

B16: Generic is too narrow and search stops early

Generic sends only the Foundation data type. All-food peanut-butter search returned a Survey (FNDDS) record that the Generic filter excludes. Branded Cheerios returned 242 matches but only 18 could be browsed. These are app restrictions, not proof that the database lacks the foods.

USDA's official guide describes search and detail endpoints and access to Foundation, FNDDS, SR Legacy, and Branded data. Use those distinctions intentionally, offer pagination, and label source types in plain language. Exact brand/barcode matches should rank differently from broad ingredient searches. [USDA API guide](#)

What is actually downloaded and connected

A roughly 459 MB FoodData_Central_csv_2026-04-30.zip exists in data/usda. The live UI does **not** search that ZIP. It calls the USDA API directly. The local README explicitly describes this arrangement. There is no indexed local catalog or offline search service in the current application.

That architecture can be reasonable, but describe it accurately: API-connected catalog plus a downloaded snapshot, not a fully integrated local database. A bulk archive on disk does not automatically create search, normalization, update handling, or shared product identity.

B22: Move private API access out of client code

The current build uses DEMO_KEY; the audit did not discover a leaked private key. However, the suggested VITE_FDC_API_KEY mechanism exposes any supplied value in client code. Vite explicitly documents this behavior. Use a server-side gateway for a private key and add environment-file ignore rules. USDA documents much lower limits for its demonstration key, including 30 requests per IP per hour and 50 per day. [Vite environment documentation](#), [USDA API guide](#)

13 / Pantry and meal suggestions: make the connection practical

The At home list is easy to start. Searching and adding a food works; quick basics and manual names offer a low-friction alternative. Adding Peanut butter and then peanut BUTTER merged into one quantity-two row. Decrementing to zero removed it. These small interactions should remain fast.

The main limitation is that quantity has no unit. One banana, one bunch of bananas, one jar, and one serving all look like a count of one. There is also no pantry consumption from logging. A USDA banana logged as eaten left inventory unchanged. The current UI asks the user to adjust quantities manually, so this is a missing connection rather than an undisclosed automatic behavior.

Start with a simple, explicit model: quantity plus unit, optional package size, storage location, and an optional low-stock threshold. Allow “Some / Low / Out” for people who do not want exact measuring. Do not force inventory bookkeeping on every meal. After logging a pantry item, offer a checked-by-choice Use from pantry control with an undoable deduction.

Meal suggestions currently filter the fixed library by schedule, access, dietary flags, cost style, and portability. They then display pantry availability. They are not ranked by how ready the pantry makes them, nor generated from an open-ended ingredient list. A ready meal can remain behind missing-ingredient suggestions because array order dominates.

Rank suitable meals by readiness and practical preparation time, while retaining variety and user favorites. Use truthful labels: Ready now, Missing 1 ingredient, Needs heating, or Pack with an ice pack. “Meals within reach” currently means at least half the ingredients match, not a meal that can actually be prepared. Rename that metric or change its definition.

Each meal needs one meaningful primary action based on state: Plan meal when ready, Add missing ingredients when incomplete, or Log this meal when eaten. Keep secondary actions available without making every card contain equal-weight commands. Add a short ingredient list and realistic preparation steps. Do not add images yet; plain, well-structured content will solve the current problem more directly.

14 / Schedule editing and calendar usability

The calendar is useful and visually understandable at desktop width. Event categories have different colors, the selected date is visible, and the agenda makes editing possible without another page. School configuration captures information that a generic calendar does not. Keep those strengths.

On a 390-pixel viewport, the month grid becomes a very small overview. Event labels are reduced to tiny, clipped fragments. The agenda and activity editor sit below the full month, so adding an event requires significant scrolling before and during entry. The calendar technically fits without document-level horizontal overflow, but fitting is not the same as being easy to use.

Proposed mobile default: a day agenda with a compact horizontal date strip. Offer Month as a view switch, not the only entry point. A prominent Add activity button should open a focused sheet. Keep school settings in a separate editable summary. The daily agenda should show school, food windows, departures, activities, and recovery opportunities in chronological order.

B17: Restore skipped sports occurrences

Skipping the September 22 occurrence of the weekly away game removed that date correctly. There was no restore affordance, unlike the school-cancellation flow. Preserve a canceled-occurrence entry or provide an Exceptions list on the series editor. Users should be able to distinguish an intentionally skipped event from an event that never existed.

Series deletion worked, but it is immediate. Introduce an undo or confirmation for deleting an entire series. Keep Delete this occurrence, Skip this date, and Delete series meaningfully different. Editing a series should clearly say whether the change affects all occurrences or only future ones. The current implementation edits the shared series.

Additional improvements: warn about overlapping activities; calculate departure time; expose recurring end dates in the summary; use full weekday names for accessibility; and rename Import school year to Save school schedule unless a real import source is being used. The current form is manual configuration, not an external calendar import.

Validation backlog: overnight sessions, daylight-saving transitions, travel across time zones, year boundaries, recurrence edits from a later occurrence, and very long names need dedicated regression coverage. They were not all executed in this audit. Do not label them passed simply because ordinary September dates worked.

15 / Today: make the first screen actionable

Today has a clear visual centerpiece: the dark recommended-action card. At 1024 pixels, the tested card and timeline were separated normally; the old overlap shown in the earlier user screenshot did not reproduce. The current .today-command has a 16-pixel bottom margin. Preserve that spacing and the wider-screen layout while making targeted changes.

The more important current problem is scale. On mobile, the brand, large personal headline, and two-row navigation push the recommended action down to approximately the middle of the first viewport. The primary action is near the bottom. On desktop, the headline and card consume considerable space while smaller timeline labels become difficult to read. Large marketing typography and tiny operational text are fighting each other.

Recommended information order

1. Compact page title and date: Today, Monday September 14.
2. One next action with its time, activity, and short reason.
3. A compact next-event/departure summary.
4. Today's meal/prep checklist.
5. Hydration quick add and recent check-in.
6. Tomorrow preparation only when relevant.

Do not restore the removed School / Next Sport / Prep card trio. A concise timeline or inline event context can convey the useful information without recreating those summary cards. Do not restore numbered workflow explanations.

The next-action card should connect to the action the user actually needs. If the meal is unplanned, Plan meal is appropriate. If it is planned but ingredients are missing, show Review missing items. If it is packed, show the next relevant step. If it was eaten, offer a check-in rather than leaving a disabled planning button as the dominant action for the rest of the window.

Reduce the explanatory paragraph to one or two short lines, with optional Why this? detail. Keep the gentle tone. Avoid using every empty state as a large colored card. An uneventful day should be calm, not visually empty but vertically long.

For the first-run state, say what setup unlocks and offer Add school and Add activity. For an intentional rest day, show a simple neutral check-in. Those states should not be conflated.

16 / A sleeker visual system

This is a proposed design specification, not a demand to replace the brand. The existing deep teal, lime, and pale neutral background are a good starting point. The next design pass should standardize their roles.

Element	Proposed rule	Why
App background	Near-white cool neutral, minimal gradient	Reduces visual haze
Main text	Deep navy/teal, such as current #12313C	Retains identity and contrast
Secondary text	A readable dark slate, not pale gray	Makes practical details usable
Primary action	Teal on light pages; lime inside dark hero	Establishes one visual priority
Success/ready	Pale green with dark text and a label	Does not rely on color alone
Warning/missing	Restrained amber/coral with text	Separates missing from destructive
Destructive action	Red text in a clearly labeled secondary action	Avoids accidental activation
Borders	One neutral border token	Removes competing outlines

Use a small spacing scale: 4, 8, 12, 16, 24, and 32 pixels. Most neighboring page sections can use 16 or 24 pixels consistently. Card padding should be 20-24 pixels on desktop and 16-20 on mobile. Use 12-16 pixel corner radii for primary surfaces and smaller radii for controls. Avoid every nested object having its own large radius and shadow.

Keep the serif accent for an onboarding or occasional editorial heading. Inside the daily application, use concise 28-36 pixel desktop page titles, 24-30 pixel mobile titles, 20-24 pixel section headings, and comfortably readable body text. These are design targets, not measured accessibility requirements. Tiny 7-11 pixel labels should not carry important food, price, or timing information.

Use shadow sparingly. A card on a card on a shadowed background makes the app look heavier, not more premium. Prefer flat list rows with dividers for inventory, groceries, logs, and events. Save raised surfaces for dialogs and focused tasks.

Standardize icons using a small code-native set. The current mix of arrows, a boxed barcode character, US circles, and miscellaneous symbols feels inconsistent. Label icon-only controls accessibly. No new meal photography or generated artwork is necessary for this phase.

17 / Navigation, responsive behavior, and keyboard access

All six primary navigation destinations were reachable at every tested width. Primary navigation also exposes `aria-current="page"`, which is a good detail to retain. No document-level horizontal overflow was measured in those sampled primary screens. This does not prove every dense or long-content state is responsive.

The navigation changes vertical position because each screen has a different introductory layout. On mobile it wraps into two rows. Food then adds another two-row control group inside a bordered card. The user has to visually distinguish primary page navigation, Food subnavigation, filters, and action buttons, all in a small area.

Recommended structure: Use one consistent compact app header. On desktop, keep the six destinations in a predictable location or use a narrow left rail if the app becomes denser. On mobile, prioritize Today, Food, and Schedule, with Weekly, History, and Profile in an accessible More destination. Alternatively retain all six in a deliberately designed menu. Do not simply squeeze more labels into the same row.

Food subsections should be a light, clearly labeled control with no background card nest. Use either a horizontal scrollable tab strip with a visible overflow cue or a single compact subsection selector. Preserve all five destinations initially; only consolidate Overview after replacing its useful summaries elsewhere.

Main view and Food subsection are component state, not URL routes. Refresh returns to Today, leaving and returning to Food resets its subsection, and browser Back does not operate as section history. Add routes such as `/today`, `/food/pantry`, `/food/groceries`, and `/food/log`. Preserve meaningful filters in the URL when useful, but never place private profile data there.

B21: Accessibility details need a focused pass

Measured prep toggles were about 23 x 23 pixels; remove controls about 22 x 23; some schedule Edit/Delete targets only 14 pixels high. Target WCAG's 24 x 24 minimum or an applicable spacing exception, and aim for 44-48 pixel touch areas for primary mobile controls. The larger target is a usability recommendation, not a claim that 44 pixels is the AA minimum. [W3C target-size guidance](#)

Food subtabs and food-type filters need programmatic selected state. Manual pantry fields need persistent labels. Search notices need live announcements. Full keyboard and screen-reader audits remain outstanding; one visible focus path was checked, not every focus sequence.

18 / Weekly, History, Profile, and small consistency issues

Weekly

The tested current period correctly reported three food check-ins, one logged day, one hydration day, a one-day logging streak, and 52 oz water. The previous period showed zero check-ins, and forward navigation was disabled at the latest period. These calculations passed the ordinary fixture.

The current logging streak remained one when browsing an older empty period. That is understandable as a current streak, but the card sits among period-specific numbers without saying so. Label it Current streak or keep it outside the period summary. Better still, prioritize planning consistency over streak pressure.

Five large statistic cards create an uneven layout: three then two on desktop, two then two then one on mobile. Reduce to two or three useful summaries and a compact detail row. The tall chart occupies considerable space for seven simple values. A shorter bar chart or seven-day strip would be easier to scan. Offer a textual data table and make internal horizontal chart scrolling obvious on narrow widths.

History

The empty state accurately explained that today's entries appear tomorrow. Controlled rollover produced the previous day's names, calories, and water. History is currently read-only and lacks date search, correction, and export. Add a date picker and a consistent food-detail editor before more analytics. Show Unknown or Not logged distinctly from zero. Avoid claiming an empty record is a meaningful completed reflection solely because a record exists.

Profile

Profile choices save and influence suggestions. Unsaved name edits are discarded when navigating away without warning. Use a clear Save changes action and a dirty-state warning, or a deliberate autosave model with visible feedback. The current Save and see today label is appropriate for initial setup but less appropriate for editing an established profile. Add a clear explanation of local storage, backup, and profile ownership here.

B18 and B23

Entering October 1 as the last grocery date during the September 14 audit persisted despite the maximum-date attribute; the summary called it Today because future differences are clamped to zero. Validate before saving, not just through input attributes. The This week and 10+ days shortcuts also save exact synthetic dates; represent uncertain recency honestly rather than pretending to know an exact trip date.

The only unexplained ordinary console error inspected was a missing favicon 404. The 429 was intentionally injected. Add the favicon as low-priority polish; do not confuse it with the more important data defects.

19 / The connected food record: implementation design

The user-facing promise should be simple: find a food once, then use it wherever it belongs. Underneath, that requires several related records, not one mutable object copied everywhere. This is a proposed architecture based on the current inconsistencies, not an already implemented system.

Record	Owns	Should not own
Food definition	Name, brand, provider IDs, nutrient basis, portions, provenance	User's remaining inventory
Ingredient relationship	Approved generic category, aliases, substitutions	Unreviewed fuzzy equality
Pantry lot	Food ID, quantity/unit, package size, location, dates	Global source nutrition edits
Grocery line	Requested food/ingredient, needed amount, price, status, origin	Final historical consumption
Purchase transaction	Bought quantity, actual price, date, pantry effect	A removable suggestion
Meal plan	Template/version, ingredients, servings, event, intended time	Completed consumption by implication
Food log entry	Eaten quantity, timestamp, source snapshot, adjustments	Live values that silently change later

Give every Nourally food a stable internal ID. Store USDA FDC IDs and Open Food Facts barcodes as provider references, not interchangeable primary keys. A barcode may identify a packaged product; a generic ingredient category may include multiple appropriate products. Neither should overwrite the other.

Store nutrient values with explicit units and basis. Preserve unknown values as null. Store source, retrieval timestamp, and relevant publication/version details. Keep household measures and package sizes separate from the 100 g nutrient basis. Version relationships when a source record changes.

For checkout, create a transaction that removes or marks the grocery line, records the purchase, and adds the correct pantry quantity together. Repeating the action should not double-stock the pantry. Undo should reverse the same transaction, not guess which rows to delete. For logging, store an immutable food snapshot and optionally create a linked pantry-consumption operation.

Migrate existing local data additively. Preserve original names, source fields, quantities, and logs even when a canonical match is uncertain. Mark those items Needs review instead of silently coercing them. Make a backup before migration, provide a schema version, and test the same migration twice to ensure it does not duplicate records.

This architecture is the prerequisite for trustworthy automation. More catalog coverage alone will not fix identity, quantity, or transaction problems.

20 / The desired end-to-end food workflow

1. Stock what is actually available

The user opens Food > At home and taps Add food. Search defaults to the pantry destination, with recent foods and basics visible before a network request. They select bread or a branded peanut butter, enter one loaf or one jar, and optionally enter an expiry date. The resulting row identifies the actual product and its usable quantity.

2. Plan against the schedule

Nourally identifies tomorrow's early away game and the departure time. It proposes a small set of suitable meals/snacks with exact ingredient requirements and preparation constraints. Ready now means every required ingredient is satisfied, not half. Missing ingredients are explicitly listed.

3. Build a grocery proposal

The user chooses which meals to plan. Nourally subtracts usable inventory and items already on the list or in the cart. It proposes only the remaining quantities, groups duplicates, preserves custom items, and marks unknown prices. The user can accept selected recommendations without replacing their whole list.

4. Shop and reconcile

As items are bought, the user can substitute a specific product, enter the actual price and package size, and mark it purchased. That creates a purchase record and updates inventory. The cart stays an in-app shopping aid unless a real retailer integration is deliberately added. Do not imply that Add to cart places an external order.

5. Prepare, eat, and log

The planned meal creates linked prep tasks. When the athlete eats it, Log meal opens a quantity confirmation using the planned ingredients. The user can change what they actually ate. If desired, the app deducts those quantities from the pantry and lets the user undo the deduction independently of correcting the historical log.

6. Review without extra chores

Food > Overview shows useful exceptions: three items running low, one meal missing an ingredient, or one unpriced grocery item. Weekly connects completed preparation and food check-ins to the schedule without grading intake. A user should not need to maintain three separate versions of what was bought, what is available, and what was eaten.

Every stage should tolerate partial information. Unknown portion, unknown price, missing barcode, and uncertain inventory are normal states. The interface should help resolve them without turning them into zeros, invented certainty, or a blocked workflow.

21 / Backend, catalog, and resilience work

The app currently combines most screens, data handling, and business rules in a 1,554-line `src/main.jsx`. This is manageable for a prototype but makes consistency fixes harder: pantry matching, meal ingredients, grocery generation, barcode normalization, and logging are implemented through separate paths.

Refactor incrementally around tested behavior. Suggested modules are schedule/time rules, food/provider normalization, inventory transactions, grocery planning, food logging, and persistence. Build shared UI components for food rows, quantity editors, status messages, and empty states. Do not begin with a wholesale rewrite that risks losing the already working flows.

Catalog service: Add a small server-side API for search/details, private-key access, caching, and provider error handling. Normalize USDA and Open Food Facts through explicit adapters. Cache stable details by provider ID and timestamp. Avoid downloading a giant catalog into the browser bundle.

Bulk snapshot option: If the downloaded USDA archive is meant to support local search, build an ingestion job, validate counts and required fields, index searchable names/brands/barcodes, expose a paginated search API, and document refresh behavior. The current ZIP plus checksum is an input artifact, not this service. The audit did not extract or validate every database row.

Storage resilience: Put a versioned validation layer between persisted data and React state. Back up before migrations. Handle malformed JSON, missing arrays, unsupported versions, quota errors, and unavailable storage gracefully. For cross-tab behavior, use explicit operations/revisions rather than repeatedly writing a stale entire object. Add an error boundary with a recoverable message.

Account choice: Either make local-only profiles a coherent product mode or implement real authenticated ownership and sync. This choice affects the persistence architecture, but neither option requires changing the core interface into a social app. Export and deletion controls should correspond to the actual storage model.

Dependencies and delivery checks: The production build passed. The declared dependencies include several latest ranges; the lockfile does pin the current install. Use intentional ranges and add lint, typecheck, unit tests, and browser smoke tests. Consider lazy-loading the barcode scanner, then measure the production result rather than assuming a bundle-size improvement. No Lighthouse or low-end-device performance claim was made in this audit.

22 / What to add, what to change, and what to defer

Add next: small features with high daily value

- **Recent and favorite foods:** avoid repeating provider searches for breakfast and common snacks.
- **One portion editor:** shared by pantry logging, search, barcode, and manual entries.
- **Undo:** for purchase, inventory use, log deletion, series deletion, and list regeneration.
- **Quantity units and low stock:** practical inventory without requiring perfect measurements.
- **Editable item prices:** with honest unknown-price accounting.
- **A meal-to-log shortcut:** plan and actual consumption should connect without being conflated.
- **Departure-aware prep:** use the travel information already collected.
- **Clear local backup/export:** protect users before building more history.

Change existing behavior before adding breadth

Make food identity deterministic, merge generated groceries, rank meals by available ingredients, preserve manual work, retain route state, fix active lunch detection, and make notifications reflect actual timing. These changes strengthen existing features; they do not require another feature category.

Make the grocery list explain its reasoning in a compact way. “For Tuesday’s away game” is more useful than repeating a generic category on every row. Show enough detail to understand a suggestion, with more available on demand. Do not make the user fill out additional feedback-style subquestions after every action.

Defer until the core is reliable

Defer an open-ended AI coach, social feeds, leaderboards, detailed body-weight targets, coach dashboards, complicated gamification, a large recipe-authoring system, and external retailer checkout. These expand complexity without resolving the current broken connections.

Also defer new photography. The app can look considerably more polished through type, spacing, hierarchy, interaction, and consistency. A missing product image should never block identifying or logging a food. If product images are later added, they should be optional and source-attributed, not the primary way to distinguish foods.

Decision rule: A proposed feature earns priority if it reduces repeated effort in the school → food → shopping → eating loop, fixes a trustworthy-data gap, or makes an existing action easier to find. If it mainly adds another dashboard card, explanation, or destination, it should wait.

23 / Implementation sequence and acceptance gates

This is an ordered development backlog, not a promised calendar estimate. Complete one gate before expanding the next.

Gate A: protect correctness and existing work

Fix B01-B08 and B22. Introduce safe parsing, schema validation, food identity rules, non-destructive list generation, truthful prices, portion provenance, and isolated profiles or explicit single-profile wording. Add regression tests for the exact reproduced cases.

Exit criteria: Peanut butter does not satisfy sunflower butter; user-added groceries survive generation; \$0 does not silently mean unlimited; unknown prices do not inflate remaining budget; a second profile cannot inherit data accidentally; cross-tab water reaches 72 in the documented test; malformed storage shows recovery rather than a blank page.

Gate B: shorten the daily interface

Fix B09, selected-state semantics, input labels, target sizes, and route persistence. Implement the compact header, lightweight Food subsection navigation, and contextual Add food. Preserve the current desktop spacing around Today. Keep all existing useful destinations reachable.

Exit criteria: At 390 pixels, Food log content appears immediately after its heading, even after searching a popular brand. Primary touch targets are comfortable. Keyboard navigation reaches all destinations and announces selected states. Refresh preserves the current route. No numbered workflow list or old three-card summary is reintroduced.

Gate C: complete food transactions

Implement pantry units, grocery-price editing, purchase transactions, optional consumption deductions, meal-plan records, and meal-to-log conversion. Add favorites/recent once identity is stable.

Exit criteria: One product can be searched, purchased, stocked, planned, eaten, and corrected without losing its identity or double-counting quantity. Every important mutation has clear feedback and an appropriate undo path.

Gate D: improve schedule intelligence

Fix lunch interval handling, multi-session context, departure timing, skipped-occurrence restoration, empty meal states, and notification wording. Review template variants and food constraints. Test real-device notification and camera behavior before changing capability claims.

Exit criteria: Defined timing fixtures produce the intended state at interval boundaries. A departed/away athlete is not treated as being at home. The app explains why no meal fits and offers a relevant correction. Real-device capabilities are explicitly documented as tested or unsupported.

24 / Functional test matrix: completed paths

This matrix summarizes the executed walkthrough. “Pass” applies to the described path, not to every possible input or environment. “Mixed” means the action worked but exposed a related defect already documented.

Area	Executed path	Result
Startup	Start local server; load fresh isolated session	Pass after starting server
Build	Production Vite build	Pass
Profile	Save name, budget, food sources, family setting	Pass
Profile	Deselect every food source	Save disabled
Profile	Vegan + gluten-free + nut-free selection	Idea set changed
Profile	Navigate away from unsaved name	Edit discarded without warning
Account	Empty email, missing first name, wrong email	Errors displayed
Account	Create, then sign in with uppercase email	Pass for local prototype
Account	New name/email with existing device data	Fail: data inherited
School	Save hours, lunch, snacks, access, commute	Pass
School	Invalid lunch and out-of-hours snack	Rejected
School	Hide/show, cancel date, restore date	Pass
Calendar	Previous/next month and date selection	Pass
Activity	Add/edit one-time practice	Pass
Activity	Add/delete low-intensity travel workout	Pass
Activity	Weekly game; edit series; skip date; delete series	Mixed: no skipped-date restore
Activity	End before start; no repeat weekday	Rejected
Today	No schedule, upcoming, quick window, active, recovery	Rendered; timing defects found
Prep	Add meal tasks, complete/uncomplete, remove	Pass
Tomorrow	Build early-event list, complete/remove tasks	Pass; limited rebuild affordance
Hydration	+8, +12, +16, +24; subtract 8	Pass
Reminders	Lead setting, evening toggle, on/off	Pass with synthetic permission
Reminders	Actual countdown text in payload	Fail: configured lead used
Pantry	USDA add twice; increment/decrement	Pass
Pantry	Manual case-insensitive duplicate; remove at zero	Pass
Pantry	Quick basic add/remove	Pass
Pantry	Log food and check remaining stock	Not connected automatically

25 / Functional test matrix: food, reliability, and limits

Area	Executed path	Result
USDA	Live Generic banana	Pass; Foundation results
USDA	Live Branded Cheerios	Pass; 242 matches, only 18 shown
USDA	Live All-food peanut butter	Pass; survey + branded results
USDA	Add one result to pantry, grocery, log	Pass with portion/price limitations
USDA	Empty/short query after search	Error; old results can remain
USDA	Controlled 429 and no-results response	Errors displayed; stale-result defect
Groceries	All five shopping goals	Lists generated
Groceries	Custom item with price	Pass until regeneration
Groceries	Regenerate with custom item and existing cart	Custom lost; cart retained
Groceries	\$5 and \$0 budgets	Fail: \$8 and \$49.25 generated
Groceries	Quantity, list/cart moves, per-item purchase	Pass
Groceries	Mark whole cart bought	Pantry/history/date updated
Groceries	Existing USDA pantry item versus generated catalog item	Duplicate recommendation
Groceries	Recency shortcuts and future exact date	Mixed; future date accepted
Meals	Add missing ingredient twice; then purchase	No duplicate; availability updated
Meals	More options and add alternate to prep	Pass; additive, not replacement
Meals	Peanut/sunflower ingredient identity fixture	Fail
Meals	No accessible source during school	Empty panel; wrong setup action
Food log	Manual required name, add, edit, delete	Pass
Food log	Optional calorie entry	Pass
Food log	Edit USDA calories without changing grams	Provenance inconsistency
Barcode	Live Nutella barcode; 15 g calculation	Pass: 81 kcal
Barcode	Short invalid number, zero serving	Rejected/disabled
Barcode	2500 g above declared max	Add remains enabled
Barcode	Controlled zero-calorie product	Fail: cannot add
Camera	Denial; synthetic preview/capture with no barcode	Error handling and cleanup passed
Weekly	Current totals; earlier period; forward limit	Pass; current-streak labeling issue
History	Empty state; controlled next-day rollover	Pass
Storage	Ordinary reload	Pass
Storage	Two tabs and malformed JSON	Fail in both cases
Layout	Six destinations at four viewport widths	No sampled document overflow

Not run: physical barcode success, real OS notifications, every browser/OS, full screen-reader navigation, prolonged offline operation, provider quota exhaustion, production auth/sync, huge user histories, timezone/DST stress, and every food/allergy combination. These are explicit remaining tests, not implicit passes.

26 / Regression suite to build from this audit

Turn the most important findings into automated checks before making broad UI changes. A lean suite will reduce the chance that a cosmetic pass reintroduces data bugs or removes useful actions.

Unit tests: ingredient identity and approved substitutions; generic versus product relationships; unknown versus zero nutrients; grams/serving conversion; budget selection with unknown prices; quantity addition and subtraction; deduplication by stable identity; time-window boundaries; recurrence exclusions; midnight date assignment; and migration idempotency.

Component tests: every form's required/invalid state; disabled action behavior; accessible field labels; selected tabs and filters; search loading/no-results/error states; a zero-calorie entry; editing a user-adjusted source value; no suitable meals; and regeneration preview that preserves manually added rows.

Browser tests: first-run setup → school → activity → next action; search → grocery → cart → purchase → pantry; pantry → meal → missing groceries → purchase → ready meal; plan → prep → log → optional inventory deduction; independent profiles; two-tab updates; corrupted-storage recovery; and route persistence after refresh.

Visual regression states: empty Today, populated Today, long activity name, restrictive profile, empty Food, 18-result search, unknown-price grocery list, long brand names, scanner open, error messages, mobile calendar editing, and a week with mixed missing/recorded days. Cover 390, 768, 1024, and 1440 pixels, then add narrower widths and zoom testing.

Real-device checks: camera permission denied/allowed/revoked; barcode under normal and poor lighting; camera stops after leaving scanner; mobile keyboard does not cover critical actions; notification permission and delivery behavior; suspended and closed-app behavior; and reduced-motion/accessibility preferences. Do not replace these with synthetic tests alone.

Measure workflow success rather than arbitrary cosmetic scores. Useful targets include: selected Food content visible without scrolling past search results; repeat food logging in a few direct actions; no hidden data replacement; every estimated value labeled; and a clear path forward when a provider or constraint fails. These are proposed acceptance targets, not measured performance statistics from users.

27 / Evidence index and source pointers

The report's screenshots are genuine captures of this audit session. Different captures intentionally show different fixtures and stages of the workflow; counts are not expected to match across all images.

Visual evidence folder: AUDIT/evidence/

- 01-onboarding.png: fresh setup and current first-run hierarchy.
- 02-today-empty.png: no-schedule Today state.
- 03-school-calendar.png: desktop school calendar.
- 04-today-scheduled.png: populated Today and preparation.
- 05-usda-generic.png: live generic food results.
- 06-grocery-budget.png: \$5 budget and \$8 list, with USDA/canonical banana duplication.
- 07-branded-search-log.png: branded-food search/log workflow.
- 08-food-mobile-top.png and 09-food-mobile-log.png: top of Food and the distant log content.
- 10-meals-desktop.png: search panel above meal suggestions and linked prep.
- 11-weekly.png: populated weekly summary.
- 12-second-account-shared-data.png: separate test account retaining earlier food counts.
- 390-*.png and 1024-*.png: six primary destinations at mobile and intermediate widths.

Reproduction helpers: account-and-storage-check.js, timing-and-edge-check.js, and responsive-and-schedule-check.js in AUDIT/. These are audit helpers for the named isolated browser session, not a complete reusable test framework. They assume the audit fixture; do not run them against a real user's data. The storage fixture in AUDIT/evidence/ contains synthetic audit data.

Implementation pointers: src/main.jsx:237 timing guidance; :350 application state; :385 reminder delivery; :465 prototype account entry; :599 shared food search; :647 food log; :745 ingredient matching; :792 USDA pantry addition; :810 USDA logging; :858 generation; :920 purchase recording; :1177 recurrence skipping; :1340 barcode scanner; :1505 Weekly; :1541 History. Line numbers refer to the source audited on September 14 and may move after edits.

src/usda.js:53 contains the API search parameters; :77 serving-calorie calculation. src/styles.css:624 caps desktop search height; :728 removes that cap on mobile. :533 contains the current Today section gap.

28 / Standards, open questions, and final recommendation

External references were used narrowly to check provider behavior, client-side environment-variable exposure, and accessibility expectations. The application findings are from the local walkthrough and source inspection, not from competitor claims or old product notes.

- [USDA FoodData Central API guide](#): supported search/detail endpoints, data types, demo-key limits, and key responsibility.
- [Vite environment variables](#): VITE_ values are exposed to client code; they are not a private-secret mechanism.
- [W3C target-size guidance](#): target dimensions and spacing exceptions.
- [W3C text-contrast guidance](#): normal text generally needs 4.5:1, with a 3:1 threshold for qualifying large text. The audit identified small/pale text and sampled computed colors, but did not certify all composited contrast pairs or WCAG conformance.

Resolve these product decisions before the relevant implementation:

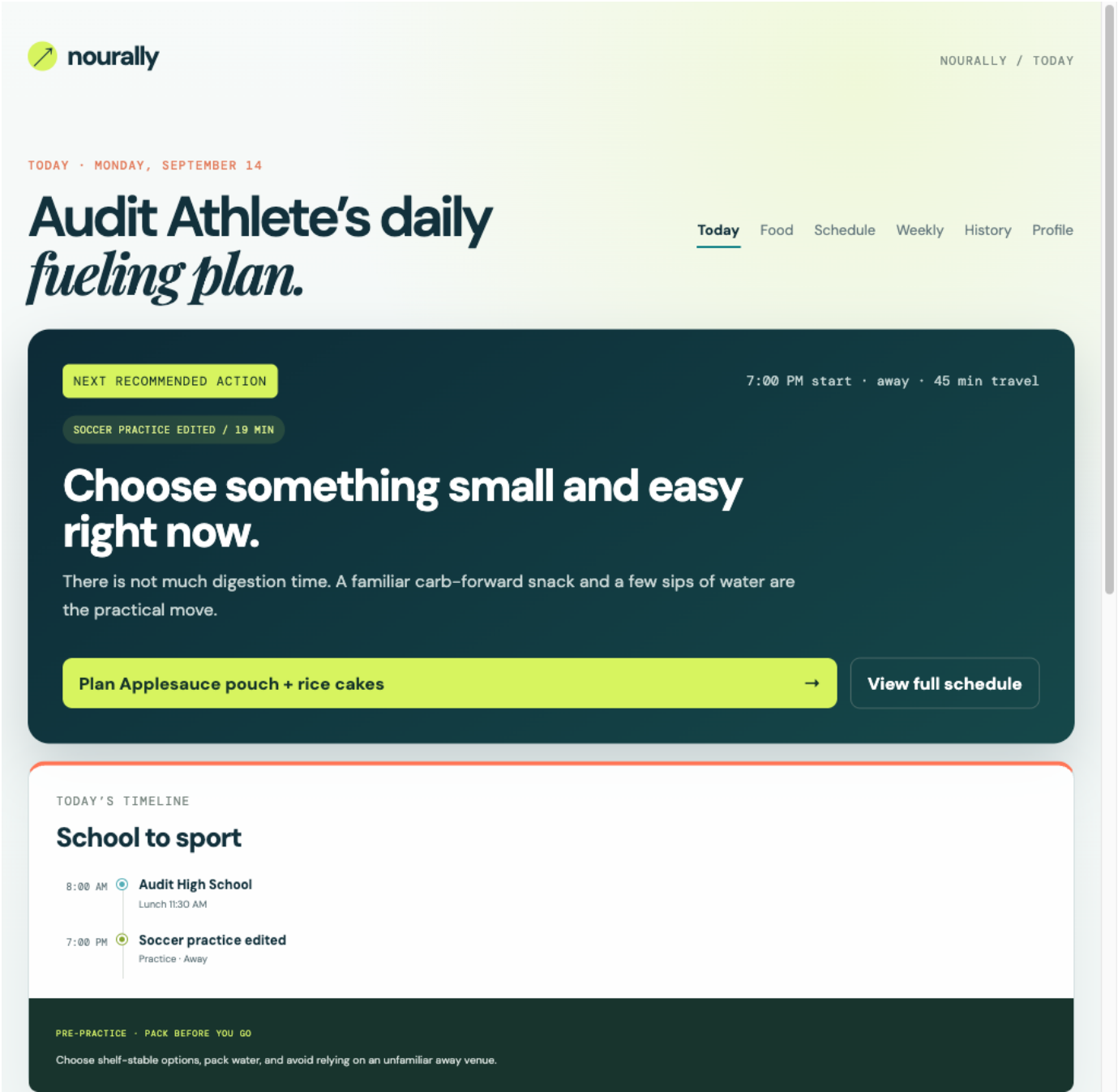
1. Is Nourally intentionally one local device profile, multiple local profiles, or a real authenticated account service?
2. Is the grocery budget per shop or per calendar week? Does zero mean zero or no limit?
3. Is pantry tracking approximate availability, exact units, or a user-selectable mix?
4. Should logging deduct inventory automatically, by explicit confirmation, or never? My recommendation is explicit, undoable confirmation.
5. Are reminders promised only while open, or will background delivery become a real supported capability?
6. Is the downloaded USDA snapshot for backup/research, or will a maintained search service ingest it?

These decisions do not block small clarity fixes. They do block trustworthy claims about privacy, remaining budget, available food, and reminder delivery.

Final recommendation: Do not add more feature categories yet. Make the existing connections truthful and fast. Keep the brand; reduce the oversized introductions; put the selected task first; preserve user work; normalize food identity and units; and test the exact cases that failed here. The result will feel more modern because it behaves coherently, not because it has more decoration.

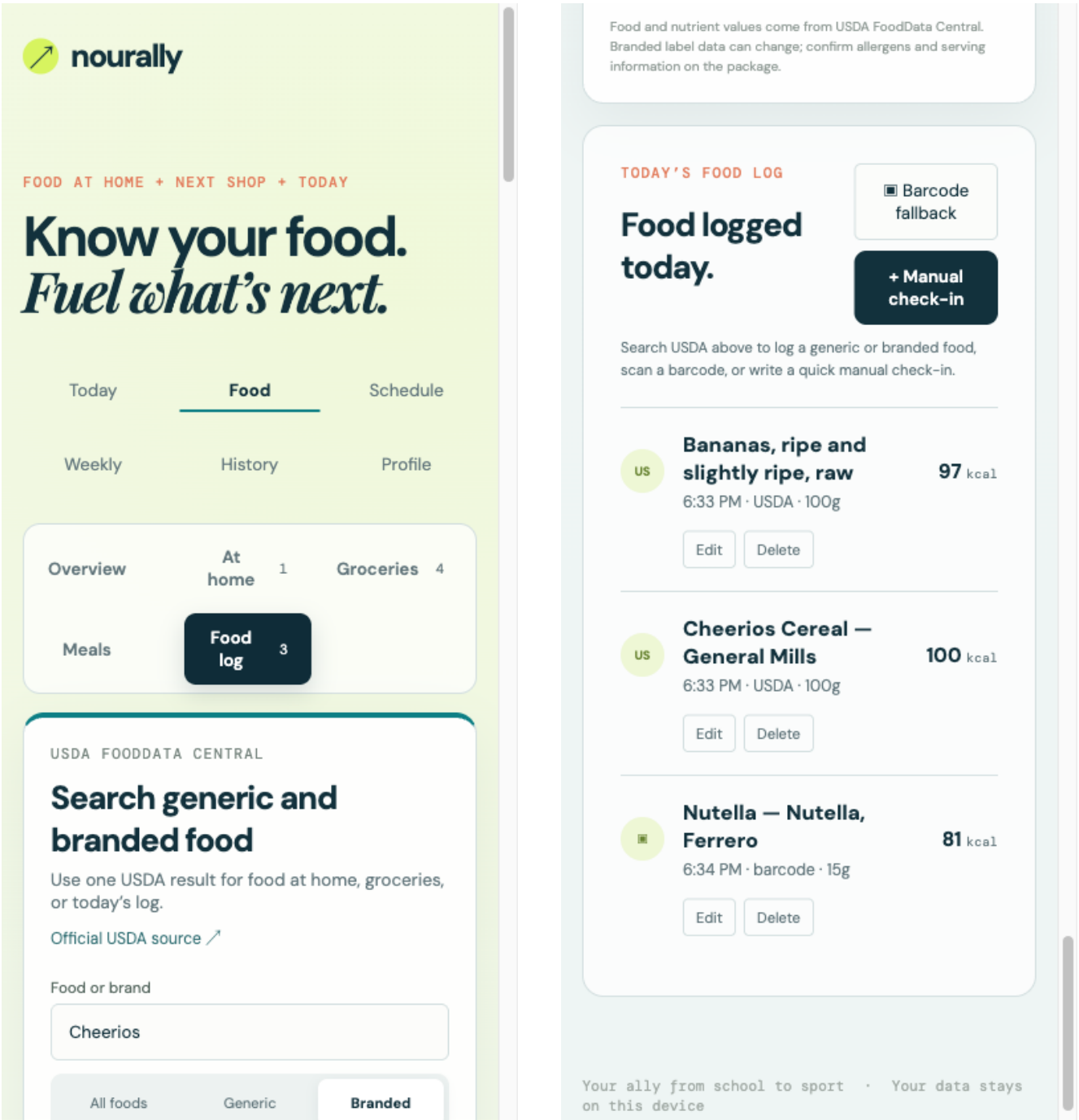
The first concrete development ticket should pair food-identity correction with its regression test. The next should make grocery generation non-destructive. In parallel, the highest-impact visual ticket is moving shared search out of the way of the selected Food subsection. Those changes directly address the weakest parts of the current experience while preserving what already works.

Visual A / Today at an intermediate width



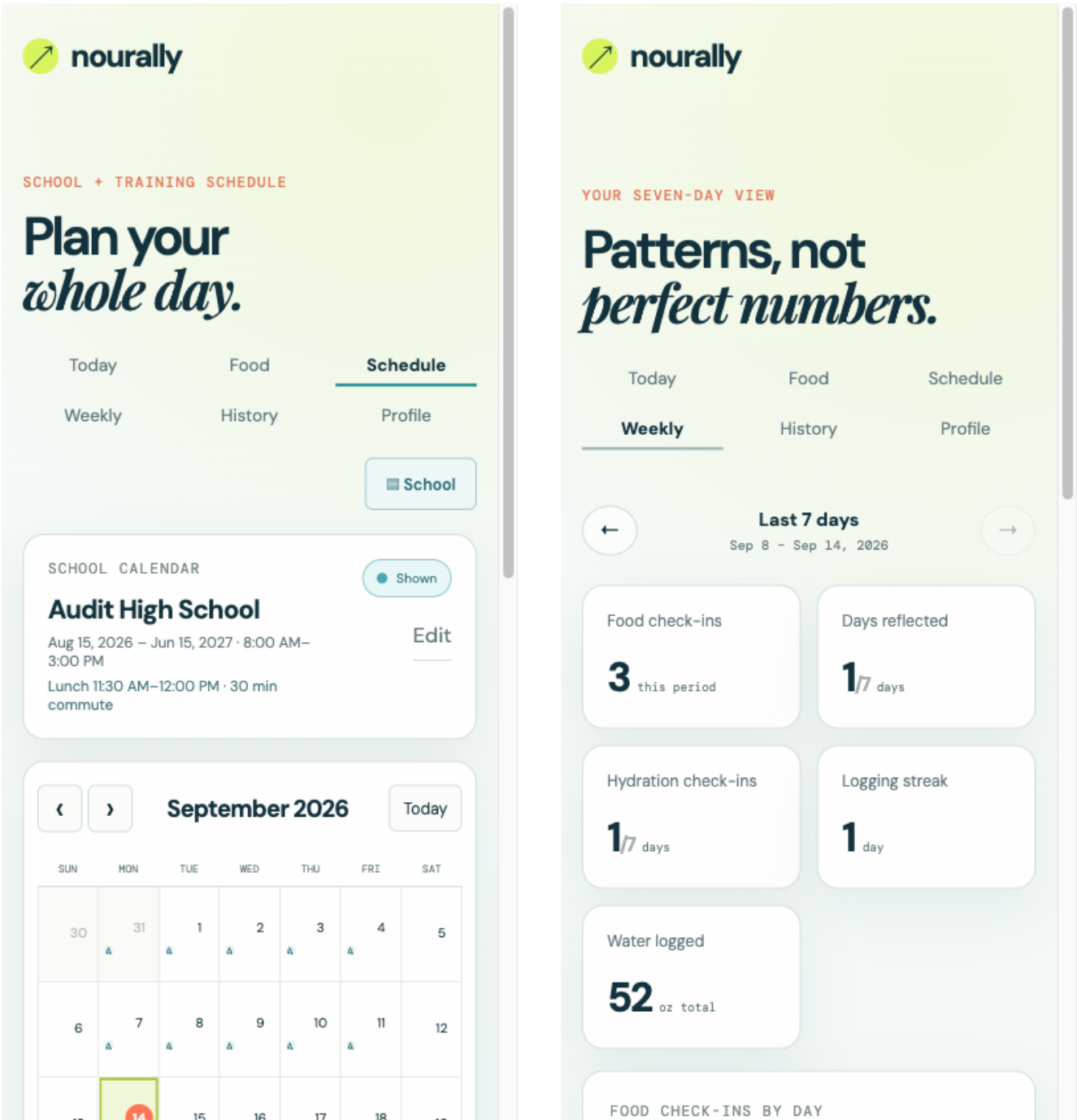
The action card and timeline are separated correctly in this tested layout. The dominant hero is clear, but large introductory typography and very small operational labels reduce the amount of useful information visible at once. Preserve the gap while improving hierarchy.

Visual B / Mobile Food: the selected task is buried



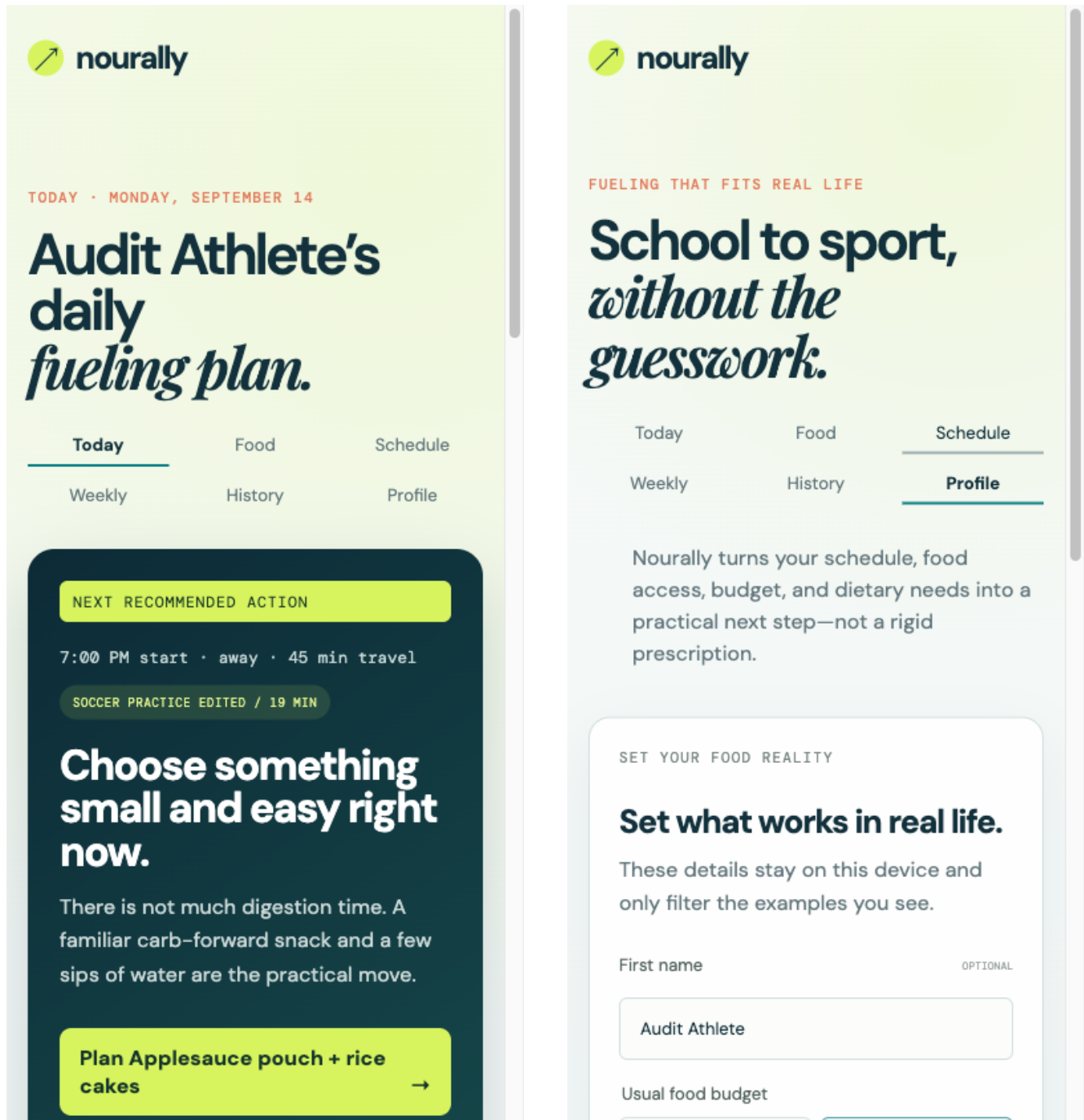
Left: the top of the Food workspace. Right: the food-log content after moving far down the document in the populated search state. The same search results remain above the selected subsection. This is the highest-impact navigation change recommended in the report.

Visual C / Mobile calendar and weekly summary



The calendar fits but its event text is extremely small. The weekly page spends most of its first viewport on the introduction and summary cards. Prefer a mobile agenda and a more compact summary; keep detailed views available.

Visual D / Mobile Today and Profile



Repeated editorial introductions consume valuable mobile space across different destinations. Use a consistent, compact application header and save the large brand expression for onboarding. No new imagery is required.